

Amazon Most Asked SQL Interview Questions

CTC - 25LPA

Table Setup:

Let's assume a table named employees with the following columns:

- employee_id (INT)
- employee_name (VARCHAR)
- salary (DECIMAL)
- department_id (INT)

Sample Data:

employee_id	employee_name	salary	department_id
1	Alice	50000	101 102 101 102
2	Bob	60000	101 101
3	Charlie	70000	
4	Dave	80000	
5	Eve	50000	
6	Frank	70000	

employee_id	employee_name	salary	department_id
78	Grace	60000	102
	Hank	80000	

1. How would you find the second highest salary in a table without using LIMIT or TOP?

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Explanation:

- We find the maximum salary that is less than the highest salary, which gives us the second-highest salary.

2. Write a query to find duplicate rows in a table and the count of their occurrences.

```
SELECT employee_name, salary, COUNT(*) AS occurrences
FROM employees
GROUP BY employee_name, salary
HAVING COUNT(*) > 1;
```

Explanation:

- This query groups the rows by employee_name and salary, and filters out only the rows that appear more than once (duplicates).

3. How would you retrieve the nth highest salary from a table?

```
SELECT DISTINCT salary AS nth_highest_salary
```

```
FROM employees
```

```
ORDER BY salary DESC
```

```
LIMIT 1 OFFSET n-1;
```

(Replace n with the desired position for the nth highest salary)

Explanation:

- The query sorts the salaries in descending order and uses LIMIT with OFFSET to fetch the nth highest salary.
- For example, to get the 3rd highest salary, replace n with 3.

4. Write a query to identify employees whose salaries are greater than the average salary in their department.

```
SELECT e.employee_name, e.salary, e.department_id
```

```
FROM employees e
```

```
WHERE e.salary > (
```

```
    SELECT AVG(salary)
```

```
    FROM employees
```

```
    WHERE department_id = e.department_id
```

```
);
```

Explanation:

- This query compares each employee's salary with the average salary of their department. Only employees whose salaries are greater than the average will be selected.

5 .How can you delete duplicate rows from a table while keeping only one instance of each?

```
WITH CTE AS (  
    SELECT employee_id, ROW_NUMBER() OVER (PARTITION BY employee_name, salary  
ORDER BY employee_id) AS row_num  
    FROM employees  
)  
DELETE FROM employees  
WHERE employee_id IN (  
    SELECT employee_id  
    FROM CTE  
    WHERE row_num > 1  
);
```

Explanation:

- This query uses a Common Table Expression (CTE) to assign a row number to each duplicate record based on employee_name and salary.
- The ROW_NUMBER() function is used to differentiate duplicates, and we delete those with a row number greater than 1.

6. Write a query to find employees who have the highest salary in each department.

```
SELECT e.employee_id, e.employee_name, e.salary, e.department_id  
FROM employees e  
WHERE e.salary = (  
    SELECT MAX(salary)  
    FROM employees  
    WHERE department_id = e.department_id  
);
```

Explanation:

- For each employee, compare their salary with the maximum salary in their department. Only employees matching the highest salary in their department are returned.
-

7. How would you retrieve records where a column contains only numeric data, even if it's stored as a string?

Assume the column is data_column in a table records.

```
SELECT *  
FROM records  
WHERE data_column REGEXP '^[0-9]+$';
```

Explanation:

- The REGEXP operator is used to check if the column contains only numeric characters (^[0-9]+\$ ensures the entire string is numeric).
-

8. Write a query to find the running total of sales in a sales table.

Assume the table sales has columns sale_date and amount.

```
SELECT sale_date, amount,  
       SUM(amount) OVER (ORDER BY sale_date) AS running_total  
FROM sales;
```

Explanation:

- The SUM() function with the OVER clause computes the running total by summing up the amount column in the order of sale_date.
-

9. How would you find the longest consecutive sequence of dates in a table?

Assume the table dates has a column date_column with unique date values.

WITH RankedDates AS (

SELECT date_column,

ROW_NUMBER() OVER (ORDER BY date_column) AS rank

FROM dates

),

GroupedDates AS (

SELECT date_column, DATE_SUB(date_column, INTERVAL rank DAY) AS grp

FROM RankedDates

)

SELECT MIN(date_column) AS start_date,

MAX(date_column) AS end_date,

COUNT(*) AS consecutive_days

FROM GroupedDates

GROUP BY grp

ORDER BY consecutive_days DESC

LIMIT 1;

Explanation:

- Assign a ROW_NUMBER() to each date in ascending order.
- Use the difference between the date_column and its ROW_NUMBER() to identify groups of consecutive dates.
- Group by this calculated value (grp), and calculate the length of each group. The longest group is the result.

10. Pivot a table's data from rows to columns

Assume the table sales has columns region, year, and sales_amount. The goal is to pivot year as columns with corresponding sales_amount.

Query:

```
SELECT region,  
  
       SUM(CASE WHEN year = 2021 THEN sales_amount ELSE 0 END) AS sales_2021,  
       SUM(CASE WHEN year = 2022 THEN sales_amount ELSE 0 END) AS sales_2022,  
       SUM(CASE WHEN year = 2023 THEN sales_amount ELSE 0 END) AS sales_2023  
FROM sales  
GROUP BY region;
```

Explanation:

- Use CASE statements to create conditional columns for each year. Sum the sales_amount for each condition to pivot the rows into columns.
-

11. How would you calculate the cumulative percentage of a column in a table? Assume the table sales has columns sale_amount and sales_id.

```
SELECT sales_id, sale_amount, (sale_amount / total_sales) * 100  
AS cumulative_percentage
```

```
FROM sales,  
  
(SELECT SUM(sale_amount) AS total_sales FROM sales) AS total  
ORDER BY sales_id;
```

Explanation:

- The cumulative percentage is calculated by dividing each sale_amount by the total sales and multiplying it by 100.
 - The subquery calculates the total sales (SUM(sale_amount)) and this value is used for the percentage calculation for each record.
-

12. Write a query to find gaps in a sequence of numbers in a table.

Assume the table numbers has a column num.

```
SELECT n1.num + 1 AS missing_number  
FROM numbers n1  
LEFT JOIN numbers n2 ON n1.num + 1 = n2.num  
WHERE n2.num IS NULL  
ORDER BY missing_number;
```

Explanation:

- This query joins the numbers table to itself (n1 and n2), where it checks for the next number (n1.num + 1) in the sequence that does not exist (n2.num IS NULL).
 - The result will give the missing numbers in the sequence.
-

13. How would you retrieve records that belong to a specific time window (e.g., last 7 days)?

Assume the table orders has a column order_date.

```
SELECT *  
FROM orders  
WHERE order_date >= CURDATE() - INTERVAL 7 DAY;
```

Explanation:

- This query uses the CURDATE() function to get the current date and subtracts 7 days (INTERVAL 7 DAY) to get the records from the last 7 days.
-

14. Write a query to join a table with itself to find employees who share the same manager.

Assume the table employees has columns employee_id, employee_name, and manager_id.

```
SELECT e1.employee_name AS employee, e2.employee_name AS manager
```

FROM employees e1

JOIN employees e2 ON e1.manager_id = e2.employee_id;

Explanation:

- The table employees is joined with itself, where e1.manager_id matches e2.employee_id. This will give the list of employees along with their managers.
-

15. How would you find the top three customers by total purchase amount in each region?

Assume the table purchases has columns customer_id, region, and purchase_amount.

SELECT region, customer_id, SUM(purchase_amount) AS total_purchase

FROM purchases

GROUP BY region, customer_id

ORDER BY region, total_purchase DESC

LIMIT 3;

Explanation:

- This query groups the data by region and customer_id to calculate the total purchase amount.
 - The results are ordered by region and total_purchase in descending order, and LIMIT 3 is used to get the top 3 customers by total purchase amount in each region.
-

16. Write a query Find the maximum difference between two consecutive values in a column

Assume the table numbers has a column value.

WITH CTE AS (

SELECT value,

LAG(value) OVER (ORDER BY value) AS prev_value

FROM numbers

)

```
SELECT MAX(value - prev_value) AS max_difference
```

```
FROM CTE
```

```
WHERE prev_value IS NOT NULL;
```

Explanation:

- Use the LAG() function to get the previous value for each row based on the order of value.
 - Calculate the difference between the current value and the previous value and find the maximum difference.
-

17. How would you Identify the median value of a column in a table

Assume the table numbers has a column value.

```
SELECT AVG(value) AS median
```

```
FROM (
```

```
  SELECT value
```

```
  FROM numbers
```

```
  ORDER BY value
```

```
  LIMIT 2 - (SELECT COUNT(*) % 2 FROM numbers) -- Pick 1 or 2 middle rows based on odd/even count
```

```
  OFFSET (SELECT (COUNT(*) - 1) / 2 FROM numbers)
```

```
) AS median_values;
```

Explanation:

- Order the values and calculate the number of rows to determine the middle value(s).
 - For an odd count, pick one middle value; for an even count, pick two middle values and take their average.
-

18. Write a Query to find overlapping date ranges in a table

Assume the table `date_ranges` has columns `range_id`, `start_date`, and `end_date`.

```
SELECT t1.range_id AS range1, t2.range_id AS range2
```

```
FROM date_ranges t1
```

```
JOIN date_ranges t2
```

```
ON t1.start_date <= t2.end_date AND t1.end_date >= t2.start_date
```

```
WHERE t1.range_id < t2.range_id;
```

Explanation:

- This query finds overlapping ranges by checking if `start_date` of one range is before or equal to the `end_date` of another range and vice versa.
 - The condition `t1.range_id < t2.range_id` avoids duplicate pairs and self-joins.
-

19. Write a query to Rank employees based on their salaries within their department

Assume the table `employees` has columns `employee_id`, `department_id`, and `salary`.

```
SELECT employee_id,
```

```
department_id,
```

```
salary,
```

```
RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rank
```

```
FROM employees;
```

Explanation:

- Use the `RANK()` function with `PARTITION BY department_id` to rank employees within each department based on their salary in descending order.
-

20. Find rows where data in a specific column repeats after a certain number of rows

Assume the table `data` has a column `value` and the repetition check is after 3 rows.

```
WITH CTE AS (  
  
    SELECT value,  
           LAG(value, 3) OVER (ORDER BY id) AS prev_value_3_rows  
    FROM data  
)  
SELECT *  
FROM CTE  
WHERE value = prev_value_3_rows;
```

Explanation:

- Use the LAG() function with an offset of 3 to fetch the value from 3 rows earlier.
- Compare the current value with the value from 3 rows ago to identify repetitions.

